

Experiment 1

Design of PN Sequences and Some “Fun” with Finite Fields

Contents

1	Experiment 1: Design of PN Sequences and Some “Fun” with Finite Fields	1
1.1	Motivation	1
1.2	Design Roadmap	2
1.3	From Polynomials over $\mathbb{R}$ to Polynomials over $\mathbb{F}_2$	2
1.4	The LFSR as a Linear Dynamical System	3
1.5	Observing the Generated Sequence	3
1.6	Where Does the Feedback Polynomial Appear?	4
1.7	Primitive Versus Irreducible Polynomials	4
1.8	A First Encounter with Finite Fields	4
1.9	Primitive Elements and Maximum-Length Sequences	5
1.10	A Glimpse of the Cayley–Hamilton Theorem	5
1.11	Questions for Exploration	5
1.12	Looking Ahead	6
1.13	Why Do We Need More Than One PN Sequence?	7
1.14	Gold Sequences: Trading Perfection for Multiplicity	7
1.15	Correlation as a Measure of Similarity	8
1.16	Binary Versus Bipolar Representation	8
1.17	Autocorrelation of Maximum-Length Sequences	9
1.18	Why Are Gold Sequences Useful?	9
1.19	Correlation Exploration Questions	9
1.20	Beyond Binary PN Sequences	10

1 Experiment 1: Design of PN Sequences and Some “Fun” with Finite Fields

1.1 Motivation

Pseudo-noise (PN) sequences occupy a unique position at the intersection of digital electronics, communication systems, linear algebra, and abstract algebra. Although generated deterministically using a small amount of digital logic, these sequences exhibit many statistical properties of random noise and therefore find widespread application in spread-spectrum communications, GPS, channel sounding, synchronization, radar, cryptography, and error-control coding.

In this laboratory, we begin with the design of a Linear Feedback Shift Register (LFSR) capable of generating a maximum-length PN sequence. Along the way, we shall discover that behind this seemingly simple digital circuit lies a rich mathematical structure involving finite fields, linear dynamical systems, companion matrices, primitive polynomials, and the Cayley–Hamilton theorem.

Rather than viewing the LFSR as merely a collection of flip-flops and XOR gates, we shall gradually reinterpret it as a linear dynamical system evolving over the finite field $\mathbb{F}_2$. This perspective not only explains why certain feedback connections produce maximal-length sequences, but also provides a beautiful bridge between digital hardware and modern algebra.

The central design question throughout this experiment is:

Given a desired PN sequence length

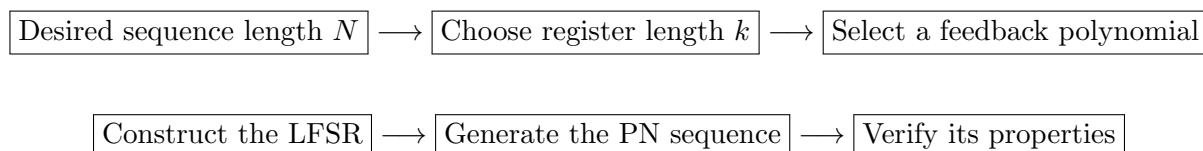
$$N = 2^k - 1,$$

how should one choose the feedback polynomial and corresponding LFSR?

The answer eventually leads us to one of the most elegant objects in finite field theory: *primitive polynomials*.

1.2 Design Roadmap

Throughout the experiment we shall follow the design flow shown below.



The principal objective is to determine which feedback polynomials generate sequences of maximum possible period and why.

1.3 From Polynomials over $\mathbb{R}$ to Polynomials over $\mathbb{F}_2$

Most of us are familiar with polynomial factorization over the real numbers.

For example,

$$x^2 + 1$$

cannot be factored over $\mathbb{R}$ since it possesses no real roots. However, over the complex numbers,

$$x^2 + 1 = (x + i)(x - i),$$

where $i^2 = -1$.

The important lesson is that the algebraic behaviour of a polynomial depends entirely upon the field over which it is considered.

In this experiment we shall instead work over the finite field

$$\mathbb{F}_2 = \{0, 1\},$$

where arithmetic is performed modulo 2. Since

$$1 = -1,$$

many familiar polynomial identities change in surprising ways.

For example,

$$x^2 + 1 = x^2 - 1 = (x + 1)^2$$

over $\mathbb{F}_2$.

One of the goals of this laboratory is to investigate which polynomials remain factorable over $\mathbb{F}_2$, which are irreducible, and eventually, which are primitive.

1.4 The LFSR as a Linear Dynamical System

Although an LFSR is usually introduced as a digital circuit, it is fundamentally a discrete-time linear dynamical system.

Suppose the contents of a four-stage shift register are

$$x[n] = \begin{bmatrix} x_1[n] \\ x_2[n] \\ x_3[n] \\ x_4[n] \end{bmatrix}.$$

Each clock pulse updates the register according to

$$x[n+1] = Ax[n],$$

where all arithmetic is performed over $\mathbb{F}_2$.

For the feedback polynomial

$$p(x) = x^4 + x + 1,$$

the corresponding state transition or **companion** matrix is

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}.$$

Thus, every LFSR may be interpreted as a linear dynamical system evolving over a finite field.

1.5 Observing the Generated Sequence

The generated PN sequence is simply a measurement of one component of the system state.

For example,

$$y[n] = Cx[n],$$

where

$$C = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix},$$

corresponds to observing the contents of the final flip-flop.

Thus the experiment naturally separates into

$$\boxed{\text{State Evolution}} \quad \longrightarrow \quad \boxed{\text{Measured PN Sequence}}.$$

Here is the complete linear dynamical system (LDS) over $\mathbb{F}_2$:

$$\boxed{\begin{aligned} x[n+1] &= Ax[n], \\ y[n] &= Cx[n]. \end{aligned}}$$

1.6 Where Does the Feedback Polynomial Appear?

An elegant observation is that the feedback polynomial of the LFSR is precisely the characteristic polynomial of the companion matrix.

Indeed,

$$\chi_A(\lambda) = \det(\lambda I - A),$$

and for the above example,

$$\chi_A(x) = x^4 + x + 1.$$

Thus the digital hardware, the state-space model, and the underlying polynomial are simply different mathematical descriptions of the same object.

1.7 Primitive Versus Irreducible Polynomials

Not every irreducible polynomial produces a maximum-length PN sequence.

This motivates one of the central questions of the experiment.

Is every irreducible polynomial suitable for PN sequence generation?

The answer is no.

Primitive polynomials form a special subset of irreducible polynomials and are precisely those that generate maximum-length sequences.

Property	Irreducible	Primitive
Cannot be factored	✓	✓
Generates field extension	✓	✓
Maximum-length LFSR	×	✓
Period equals $2^k - 1$	Not always	Always

One of the design objectives of this experiment is to compare experimentally the sequences produced by primitive and non-primitive feedback polynomials.

1.8 A First Encounter with Finite Fields

Just as adjoining the imaginary unit

$$i, \quad i^2 = -1,$$

extends the real numbers to the complex field, adjoining a root

$$\alpha$$

of an irreducible polynomial extends $\mathbb{F}_2$ to a larger finite field.

For example, if

$$\alpha^4 + \alpha + 1 = 0,$$

then

$$\alpha^4 = \alpha + 1.$$

Consequently every element of the extended field can be written uniquely as

$$a_0 + a_1\alpha + a_2\alpha^2 + a_3\alpha^3,$$

where each coefficient belongs to $\mathbb{F}_2$.

Since each coefficient has two possible values, the resulting field contains

$$2^4 = 16$$

elements.

1.9 Primitive Elements and Maximum-Length Sequences

Among all elements of the finite field, certain special elements generate every nonzero field element through repeated multiplication.

Such elements are called *primitive elements*.

If α is primitive,

$$1, \alpha, \alpha^2, \dots, \alpha^{2^k-2}$$

enumerate every nonzero element of the field exactly once.

Remarkably, a primitive polynomial produces an LFSR whose successive states correspond precisely to these successive powers of α .

Thus the shift register is, in effect, walking through the finite field one element at a time.

1.10 A Glimpse of the Cayley–Hamilton Theorem

The Cayley–Hamilton theorem states that every square matrix satisfies its own characteristic polynomial.

Consequently,

$$p(A) = 0,$$

where $p(x)$ is the feedback polynomial.

For example, if

$$p(x) = x^4 + x + 1,$$

then

$$A^4 + A + I = 0,$$

or equivalently,

$$A^4 = A + I.$$

This identity explains why the feedback recursion of an LFSR can always be expressed using only a finite number of previous register states.

1.11 Questions for Exploration

Throughout the experiment, think carefully about the following questions.

1. Why is the all-zero register state never left once entered?
2. Why can an n -stage LFSR never visit all 2^n possible states?
3. Why do primitive polynomials produce maximum-length sequences?
4. Can an irreducible polynomial fail to generate a maximal sequence?

5. Why do different nonzero initial conditions generate only shifted versions of the same PN sequence?
6. What happens experimentally if a reducible polynomial is used?
7. How does the companion matrix encode the feedback logic of the LFSR?
8. Why is the generated PN sequence periodic?
9. Can the behaviour of the LFSR be predicted entirely from its characteristic polynomial?
10. In what sense is a PN sequence “random,” and in what sense is it completely deterministic?

1.12 Looking Ahead

By the end of this experiment, the LFSR should no longer appear as merely a shift register with a few XOR gates. Instead, it may be viewed simultaneously as

- a digital sequential circuit,
- a linear dynamical system over $\mathbb{F}_2$,
- a companion matrix,
- a polynomial recurrence,
- an orbit generated by repeated matrix multiplication,
- and, ultimately, successive powers of a primitive element in a finite field.

This interplay between digital hardware and abstract algebra is one of the principal reasons that LFSRs continue to play a central role in modern communication systems, coding theory, cryptography, and digital signal processing.

1.13 Why Do We Need More Than One PN Sequence?

A maximum-length PN sequence generated by a primitive polynomial possesses a remarkable autocorrelation property. However, practical communication systems rarely involve only a single transmitter.

Consider a multiple-access communication system in which several users transmit simultaneously. Each user requires a unique spreading sequence so that the receiver can distinguish one signal from another.

This creates two competing requirements:

1. Each sequence should correlate strongly with itself.
2. Different sequences should have very small correlation with one another.

The first requirement is described by *autocorrelation*, while the second is described by *cross-correlation*.

An ideal communication sequence would therefore satisfy

$$R_{aa}[0] \gg R_{aa}[k], \quad k \neq 0,$$

and

$$R_{ab}[k] \approx 0,$$

for different sequences $a[n]$ and $b[n]$.

This requirement motivates the construction of Gold sequences.

1.14 Gold Sequences: Trading Perfection for Multiplicity

Maximum-length sequences (m-sequences) generated by primitive polynomials have excellent autocorrelation properties. However, for a register of length k , there are only a limited number of such sequences.

Gold sequences provide a method for generating a much larger family of sequences with controlled correlation properties.

A Gold sequence is obtained by modulo-2 addition of two m-sequences having

- the same length,
- the same clock source,
- carefully chosen primitive polynomials.

If the two PN sequences are

$$a[n]$$

and

$$b[n],$$

then the Gold sequence is

$$g[n] = a[n] \oplus b[n].$$

The operation $\oplus$ represents addition modulo two.

Although the resulting Gold sequence no longer possesses the perfect two-valued autocorrelation of an m-sequence, it has an important advantage:

A large number of sequences can be generated while maintaining bounded cross-correlation between different users.

This trade-off is fundamental in practical spread-spectrum communication systems such as CDMA.

1.15 Correlation as a Measure of Similarity

The correlation function measures the similarity between two sequences.

For two periodic sequences $a[n]$ and $b[n]$, the cross-correlation is defined as

$$R_{ab}[k] = \sum_{n=0}^{N-1} a[n]b[n+k],$$

where the shift is performed cyclically.

When the two sequences are identical,

$$a[n] = b[n],$$

the correlation becomes the autocorrelation:

$$R_{aa}[k] = \sum_{n=0}^{N-1} a[n]a[n+k].$$

The shift k determines the relative alignment between the two sequences.

1.16 Binary Versus Bipolar Representation

Although PN sequences are generated as binary sequences,

$$s[n] \in \{0, 1\},$$

correlation analysis is usually performed using a bipolar representation:

$$0 \rightarrow -1, \quad 1 \rightarrow +1.$$

The sequence is then written as

$$a[n] \in \{-1, +1\}.$$

The autocorrelation becomes

$$R_{aa}[k] = \sum_{n=0}^{N-1} a[n]a[n+k].$$

This representation is useful because

$$(+1)(+1) = 1,$$

$$(-1)(-1) = 1,$$

and

$$(+1)(-1) = -1.$$

Therefore, matches contribute positively and mismatches contribute negatively. The correlation function becomes a true measure of sequence similarity.

1.17 Autocorrelation of Maximum-Length Sequences

A maximum-length sequence generated by a primitive polynomial has the special property that its normalized autocorrelation contains only two values.

For

$$N = 2^k - 1,$$

the autocorrelation is

$$R_{aa}[m] = \begin{cases} 1, & m = 0, \\ -\frac{1}{N}, & m \neq 0. \end{cases}$$

Therefore, the autocorrelation consists of

- a large peak when the sequence is perfectly aligned,
- a very small value for all other cyclic shifts.

This resembles the behaviour of white noise and allows reliable synchronization even in the presence of channel noise.

1.18 Why Are Gold Sequences Useful?

The ideal autocorrelation of an m-sequence is not sufficient for a multi-user system. Several users require several sequences.

Gold sequences intentionally relax the autocorrelation requirement in order to obtain a much larger sequence family.

The comparison can be summarized as:

	m-sequence	Gold sequence
Generation	Single LFSR	XOR of two LFSRs
Autocorrelation	Two-valued	Multi-valued
Number of sequences	Limited	Large family
Multi-user communication	Difficult	Suitable

The engineering principle is therefore:

Perfect self-correlation is less important than controlled interference between many users.

1.19 Correlation Exploration Questions

During the experiment, investigate:

1. Why does an m-sequence have a sharp autocorrelation peak?
2. Why does the autocorrelation of a Gold sequence contain more than two values?
3. Can two different primitive polynomials generate sequences with low cross-correlation?
4. What happens to correlation properties when a non-primitive polynomial is used?
5. Why is a slightly imperfect Gold sequence more useful than a perfect m-sequence in a CDMA system?
6. What changes if the binary alphabet is extended to a ternary or complex alphabet?

1.20 Beyond Binary PN Sequences

The binary alphabet

$$\{-1, +1\}$$

is only one possible choice.

More general communication sequences may use larger alphabets such as

$$\{-1, 0, +1\}$$

or complex phase alphabets,

$$\{1, e^{j2\pi/3}, e^{j4\pi/3}\}.$$

Increasing the alphabet size provides additional freedom to design sequences with improved correlation properties.

This leads naturally to modern synchronization sequences such as polyphase sequences and constant-amplitude zero-autocorrelation (CAZAC) sequences.

Thus, the study of PN sequences provides the first glimpse of a broader design question:

How should one design deterministic sequences that appear random, synchronize efficiently, and interfere minimally with one another?